

SpaceX Falcon 9 First Stage Landing Prediction

Data Science Capstone Project

Prepared by: Amisha Mishra

GitHub Repository:

<https://github.com/amishamishra0006/SpaceX>

Executive Summary

Project Objective

The objective of this project is to predict whether the Falcon 9 first stage will successfully land after launch.

Data Sources

- SpaceX REST API
- Wikipedia Launch Records

Methodology

- Data Collection
- Data Wrangling
- Exploratory Data Analysis
- SQL Analysis
- Interactive Visualization
- Predictive Machine Learning

Key Results

- Multiple machine learning models were trained and evaluated.
 - The best-performing model achieved the highest classification accuracy.
 - The model can assist in predicting future Falcon 9 landing outcomes.
-

Introduction

Background

SpaceX significantly reduces launch costs by recovering and reusing Falcon 9 first-stage boosters.

Business Problem

Predict whether a Falcon 9 first-stage landing will be successful based on launch characteristics.

Project Goal

Develop a machine learning model capable of predicting landing success.

Data Collection – SpaceX API

```
graph TD; A[SpaceX REST API] --> B[Send API Request (Python Requests)]; B --> C[Retrieve Launch Data (JSON Format)]; C --> D[Extract Relevant Fields]; D --> E[Convert JSON to Pandas DataFrame]; E --> F[Data Cleaning & Feature Selection]; F --> G[Final Dataset for Analysis];
```

Data Source

SpaceX REST API

Process

1. Send API requests
2. Retrieve launch records
3. Convert JSON data into DataFrames
4. Select relevant launch features

Features Collected

- Flight Number
- Launch Site
- Payload Mass
- Orbit Type
- Booster Version
- Landing Outcome

Include:

- API workflow diagram
- Screenshot of API dataframe output

GitHub URL:

<https://github.com/khoraiankita/SpaceX-Falcon9-Landing-Prediction>

Data Collection – Web Scraping

```
[11]: # use requests.get() method with the provided static_url and headers
      # assign the response to a object
      response = requests.get(
          static_url,
          headers={"User-Agent": "Mozilla/5.0"}
      )

      print(response.status_code)
      print(response.url)

      200
      https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922

      print(column_names)

      ['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch
      outcome']

13]: # Use soup.title attribute
      print(soup.title)

      <title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>
```

```
] df = pd.DataFrame({key:pd.Series(value) for key, value in launch_dict.items()})
df.head()
```

	Flight No.	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Version Booster	Booster landing	Date
0	1	CCSFS	Transporter-1	~5,000 kg	SSO	SpaceX	Success\n	F9 B5B1058.5613	Success	24 January 2021
1	1	CCAFS	Dragon Spacecraft Qualification Unit	0	LEO	NASA(COTS)NRO	Success\n	F9 v1.07B0003.18	Failure	4 June 2010
2	2	CCAFS	Dragon	0	LEO	NASA(COTS)	Success	F9 v1.07B0003.18	Failure	4 June 2010
3	3	CCAFS	Dragon	525 kg	LEO	NASA(CRS)	Success	F9 v1.07B0004.18	No attempt\n	8 December 2010
4	4	CCAFS	SpaceX CRS-1	4,700 kg	LEO	NASA(CRS)	Success\n	F9 v1.07B0005.18	No attempt	22 May 2012

Wikipedia Page



requests.get()



BeautifulSoup



Extract Tables



Parse Rows



Create DataFrame



Export CSV

Data Source

Wikipedia Falcon 9 Launch Records

Process

1. Retrieve webpage
2. Parse HTML tables
3. Extract launch information
4. Convert data into DataFrame

Benefits

Provides additional launch information unavailable from the API.

Include:

- Web scraping workflow
- Screenshot of scraped dataframe

GitHub URL:

<https://github.com/khoraiankita/SpaceX-Falcon9-Landing-Prediction>

Data Wrangling

```
[3]: df=pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetw
df.head(10)
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False
5	6	2014-01-06	Falcon 9	3325.000000	GTO	CCAFS SLC 40	None None	1	False	False	False
6	7	2014-04-18	Falcon 9	2296.000000	ISS	CCAFS SLC 40	True Ocean	1	False	False	True
7	8	2014-07-14	Falcon 9	1316.000000	LEO	CCAFS SLC 40	True Ocean	1	False	False	True
8	9	2014-08-05	Falcon 9	4535.000000	GTO	CCAFS SLC 40	None None	1	False	False	False
9	10	2014-09-07	Falcon 9	4428.000000	GTO	CCAFS SLC 40	None None	1	False	False	False

```
[6]: # Apply value_counts() on column LaunchSite  
df['LaunchSite'].value_counts()
```

```
[6]: LaunchSite  
      CCAFS SLC 40      55  
      KSC LC 39A      22  
      VAFB SLC 4E      13  
      Name: count, dtype: int64
```

```
[7]: # Apply value_counts on Orbit column  
df['Orbit'].value_counts()
```

```
[7]: Orbit  
      GTO      27  
      ISS      21  
      VLEO     14  
      PO       9  
      LEO       7  
      SSO       5  
      MEO       3  
      ES-L1     1  
      HEO       1  
      SO        1  
      GEO       1  
      Name: count, dtype: int64
```

```
[8]: # landing_outcomes = values on Outcome column  
      landing_outcomes = df['Outcome'].value_counts()  
      landing_outcomes
```

```
[8]: Outcome  
      True ASDS      41  
      None None      19  
      True RTLS      14  
      False ASDS      6  
      True Ocean      5  
      False Ocean      2  
      None ASDS      2  
      False RTLS      1  
      Name: count, dtype: int64
```

```
[13]: df['Class']=landing_class  
      df[['Class']].head(8)
```

```
[13]:
```

	Class
0	0
1	0
2	0
3	0
4	0
5	0
6	1
7	1

```
[14]: df.head(5)
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False

We can use the following line of code to determine the success rate:

```
[15]: df["Class"].mean()
```

```
[15]: np.float64(0.6666666666666666)
```

```
[5]: df.dtypes
```

```
[5]: FlightNumber      int64
     Date              str
     BoosterVersion    str
     PayloadMass       float64
     Orbit             str
     LaunchSite        str
     Outcome           str
     Flights           int64
     GridFins          bool
     Reused            bool
     Legs              bool
     LandingPad        str
     Block             float64
     ReusedCount       int64
     Serial            str
     Longitude         float64
     Latitude          float64
     dtype: object
```


Objectives

- Handle missing values
- Clean inconsistent records
- Convert data types
- Create target variable (Class)

Data Preparation

- Removed unnecessary columns
- Processed categorical variables
- Prepared data for machine learning

Outcome

Created a clean dataset suitable for analysis and modeling.

Include:

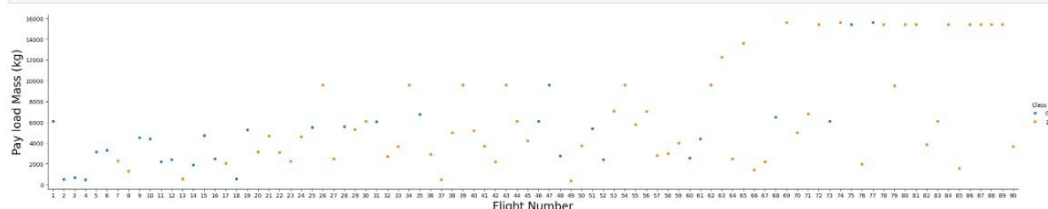
- Before and after cleaning examples

GitHub URL:

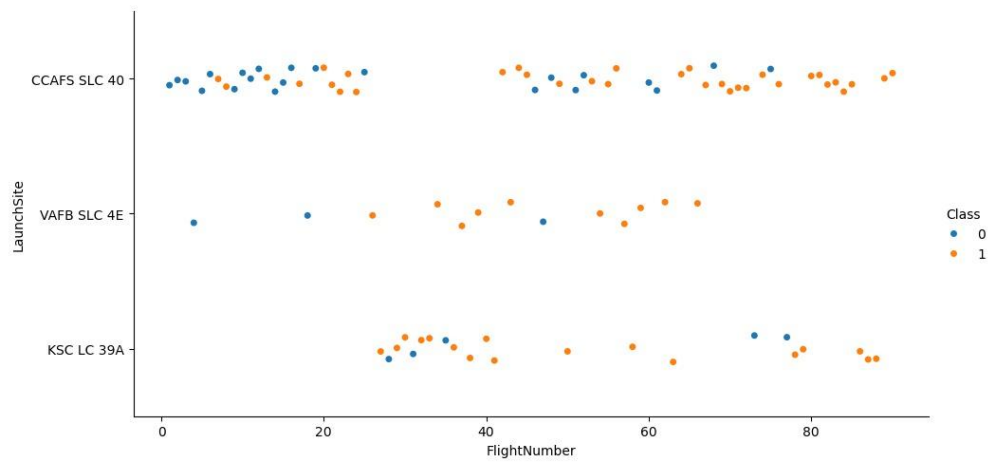
<https://github.com/khoraiankita/SpaceX-Falcon9-Landing-Prediction>

Exploratory Data Analysis (Visualization)

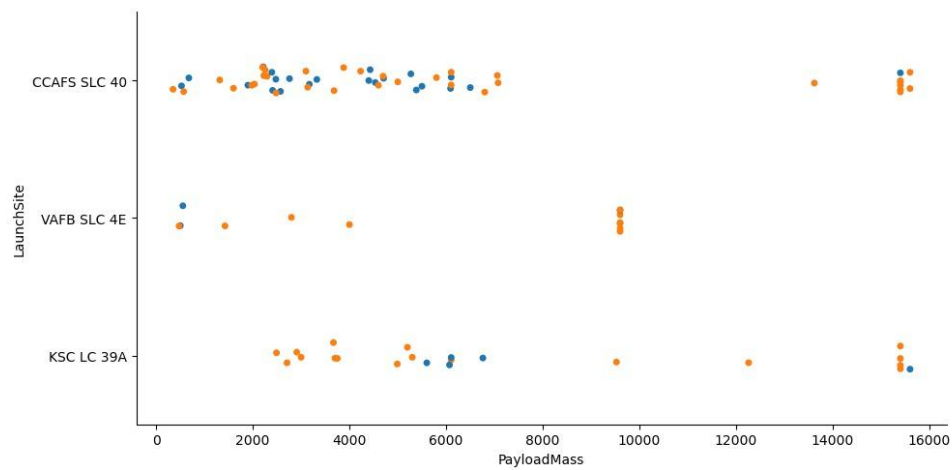
```
[4]: sns.catplot(y="PayloadMass", x="FlightNumber", hue="Class", data=df, aspect = 5)
plt.xlabel("Flight Number",fontsize=20)
plt.ylabel("Pay load Mass (kg)",fontsize=20)
plt.show()
```



```
[5]: # Plot a scatter point chart with x axis to be Flight Number and y axis to be the launch site, and hue to be the class value
sns.catplot(
    y="LaunchSite",
    x="FlightNumber",
    hue="Class",
    data=df,
    aspect=2
)
plt.show()
```

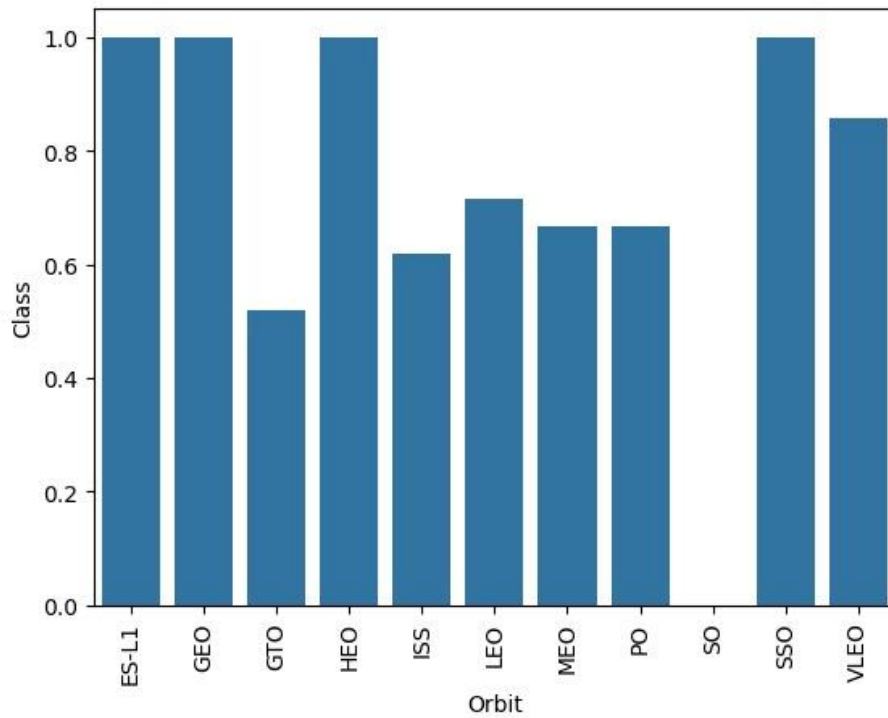


```
[6]: # Plot a scatter point chart with x axis to be Pay Load Mass (kg) and y axis to be the Launch site, and hue to be the class value
sns.catplot(
    y="LaunchSite",
    x="PayloadMass",
    hue="Class",
    data=df,
    aspect=2
)
plt.show()
```

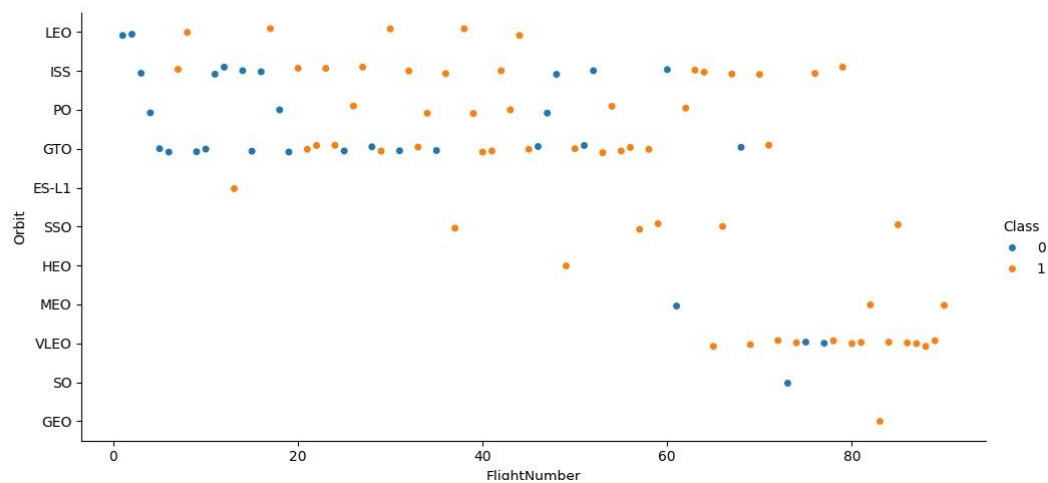


```
[7]: # HINT use groupby method on Orbit column and get the mean of Class column
orbit_success = df.groupby('Orbit')['Class'].mean().reset_index()

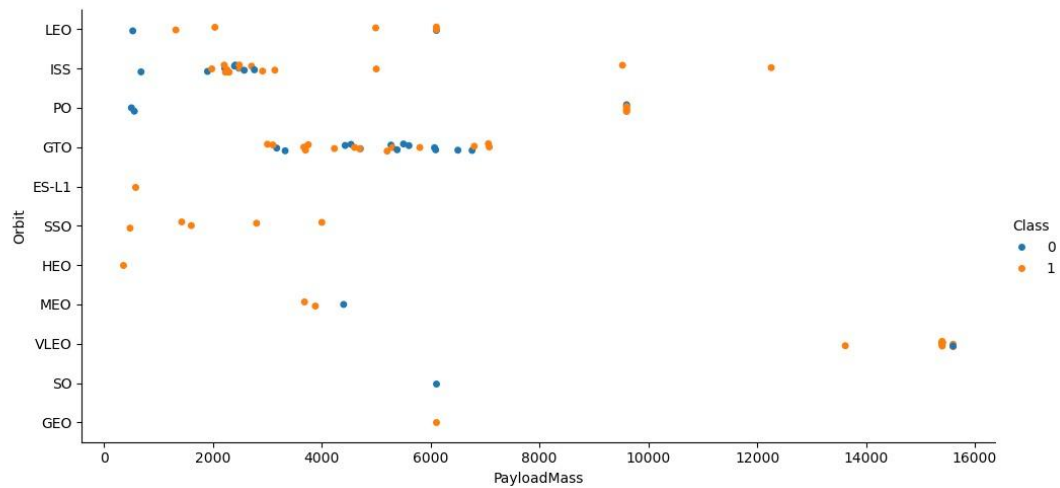
sns.barplot(x='Orbit', y='Class', data=orbit_success)
plt.xticks(rotation=90)
plt.show()
```



```
[8]: # Plot a scatter point chart with x axis to be FlightNumber and y axis to be the Orbit, and hue to be the class value
sns.catplot(
    y="Orbit",
    x="FlightNumber",
    hue="Class",
    data=df,
    aspect=2
)
plt.show()
```



```
[9]: # Plot a scatter point chart with x axis to be Payload Mass and y axis to be the Orbit, and hue to be the class value
sns.catplot(
    y="Orbit",
    x="PayloadMass",
    hue="Class",
    data=df,
    aspect=2
)
plt.show()
```



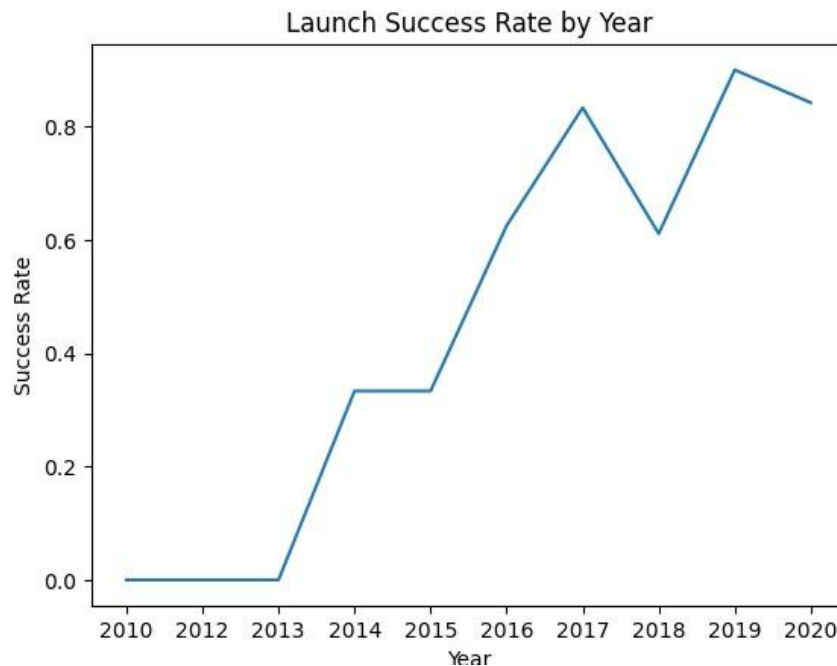
```
[10]: # A function to Extract years from the date
year=[]
def Extract_year():
    for i in df["Date"]:
        year.append(i.split("-")[0])
    return year
Extract_year()
df["Date"] = year
df.head()
```

```
[10]:
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude	Class
0	1	2010	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857	0
1	2	2012	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857	0
2	3	2013	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857	0
3	4	2013	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093	0
4	5	2013	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857	0

```
[11]: # Plot a line chart with x axis to be the extracted year and y axis to be the success rate
success_rate = df.groupby('Date')['Class'].mean().reset_index()

sns.lineplot(x='Date', y='Class', data=success_rate)
plt.xlabel("Year")
plt.ylabel("Success Rate")
plt.title("Launch Success Rate by Year")
plt.show()
```



```
[13]: # HINT: Use get_dummies() function on the categorical columns
features_one_hot = pd.get_dummies(
    features,
    columns=['Orbit', 'LaunchSite', 'LandingPad', 'Serial']
)
features_one_hot.head()
```

```
[13]:
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	...	Serial_B1048	Serial_B1049	Serial_B1050	Serial_B1051	Serial_B1054	Serial_B1056	Serial_B1058	Ser
0	1	6104.959412	1	False	False	False	1.0	0	False	False	...	False	False	False	False	False	False	False	False
1	2	525.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False	False	False	False	False
2	3	677.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False	False	False	False	False
3	4	500.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False	False	False	False	False
4	5	3170.000000	1	False	False	False	1.0	0	False	False	...	False	False	False	False	False	False	False	False

5 rows × 80 columns

```
[14]: # HINT: use astype function
features_one_hot = features_one_hot.astype("float64")
features_one_hot.head()
```

```
[14]:
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	...	Serial_B1048	Serial_B1049	Serial_B1050	Serial_B1051	Serial_B1054	Serial_B1056	Serial_B1058	Ser
0	1.0	6104.959412	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	2.0	525.000000	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	3.0	677.000000	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	4.0	500.000000	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	5.0	3170.000000	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 80 columns

Visualizations Performed

Launch Site vs Success Rate

Identified launch sites with the highest success percentages.

Payload Mass vs Landing Outcome

Observed relationship between payload weight and landing success.

Flight Number vs Landing Outcome

Success rates generally improved over time.

Yearly Trends

Launch success increased as SpaceX gained operational experience.

Include:

- Scatter plots
- Bar charts
- Trend charts

GitHub URL:

<https://github.com/khoraiankita/SpaceX-Falcon9-Landing-Prediction>

Exploratory Data Analysis (SQL)

```
[10]: %%sql

SELECT DISTINCT Launch_Site
FROM SPACEXTABLE;

* sqlite:///my_data1.db
Done.
```

[10]: **Launch_Site**

CCAFS LC-40

VAFB SLC-4E

KSC LC-39A

CCAFS SLC-40

```
[11]: %%sql

SELECT *
FROM SPACEXTABLE
WHERE Launch_Site LIKE 'CCA%'
LIMIT 5;

* sqlite:///my_data1.db
Done.
```

[11]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight	525	LEO (ISS)	NASA (COTS)	Success

2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success

[12]: %%sql

```
SELECT SUM(PAYLOAD_MASS_KG_) AS TOTAL_PAYLOAD
FROM SPACEXTABLE
WHERE CUSTOMER = 'NASA (CRS)';
```

```
* sqlite:///my_data1.db
Done.
```

[12]: TOTAL_PAYLOAD

45596

[13]: %%sql

```
SELECT AVG(PAYLOAD_MASS_KG_) AS AVG_PAYLOAD
FROM SPACEXTABLE
WHERE Booster_Version = 'F9 v1.1';
```

```
* sqlite:///my_data1.db
Done.
```

[13]: AVG_PAYLOAD

2928.4

[14]: %%sql

```
SELECT MIN(Date) AS First_Successful_Landing
FROM SPACEXTABLE
WHERE "Landing_Outcome" LIKE 'Success (ground pad)%';
```

* sqlite:///my_data1.db

Done.

[14]: **First_Successful_Landing**

2015-12-22

[15]: %%sql

```
SELECT DISTINCT Booster_Version
FROM SPACEXTABLE
WHERE "Landing_Outcome" LIKE 'Success (drone ship)%'
AND PAYLOAD_MASS_KG_ > 4000
AND PAYLOAD_MASS_KG_ < 6000;
```

* sqlite:///my_data1.db

Done.

[15]: **Booster_Version**

F9 FT B1022

F9 FT B1026

F9 FT B1021.2

F9 FT B1031.2

[16]: %%sql

```
SELECT "Mission_Outcome", COUNT(*) AS Total
FROM SPACEXTABLE
GROUP BY "Mission_Outcome";
```

* sqlite:///my_data1.db

Done.

[16]:

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

[17]: %%sql

```
SELECT DISTINCT Booster_Version
FROM SPACEXTABLE
WHERE PAYLOAD_MASS__KG_ = (
    SELECT MAX(PAYLOAD_MASS__KG_)
    FROM SPACEXTABLE
);
```

* sqlite:///my_data1.db

Done.

[17]: **Booster_Version**

F9 B5 B1048.4

F9 B5 B1049.4

F9 B5 B1051.3

F9 B5 B1056.4

F9 B5 B1048.5

F9 B5 B1051.4

F9 B5 B1049.5

F9 B5 B1060.2

F9 B5 B1058.3

F9 B5 B1051.6

F9 B5 B1060.3

F9 B5 B1049.7

```
[18]: %%sql

SELECT substr(Date,6,2) AS Month,
       "Landing_Outcome",
       Booster_Version,
       Launch_Site
FROM SPACEXTABLE
WHERE substr(Date,1,4)='2015'
AND "Landing_Outcome" LIKE 'Failure (drone ship)%';

* sqlite:///my_data1.db
Done.
```

```
[18]:
```

	Month	Landing_Outcome	Booster_Version	Launch_Site
	01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
	04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

```
[19]: %%sql

SELECT "Landing_Outcome",
       COUNT(*) AS Count
FROM SPACEXTABLE
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY "Landing_Outcome"
ORDER BY Count DESC;

* sqlite:///my_data1.db
Done.
```

```
[19]:
```

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

SQL Queries Performed

Launch Site Analysis

Identified launch frequencies by site.

Payload Analysis

Examined payload distributions.

Success Rate Analysis

Calculated landing success percentages.

Ranking Queries

Compared launch sites and mission outcomes.

Time-Based Analysis

Analyzed launch trends across years.

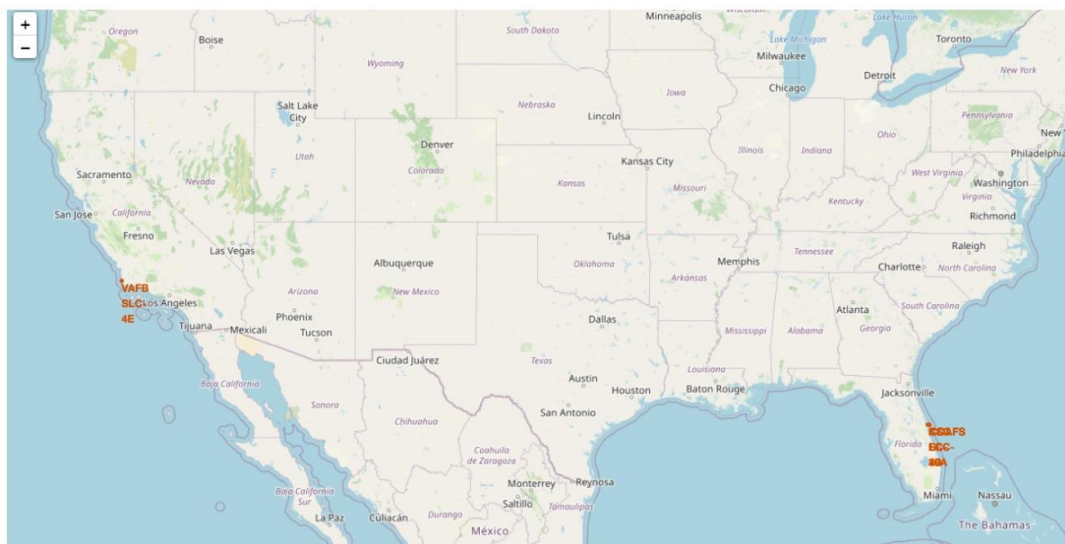
Include:

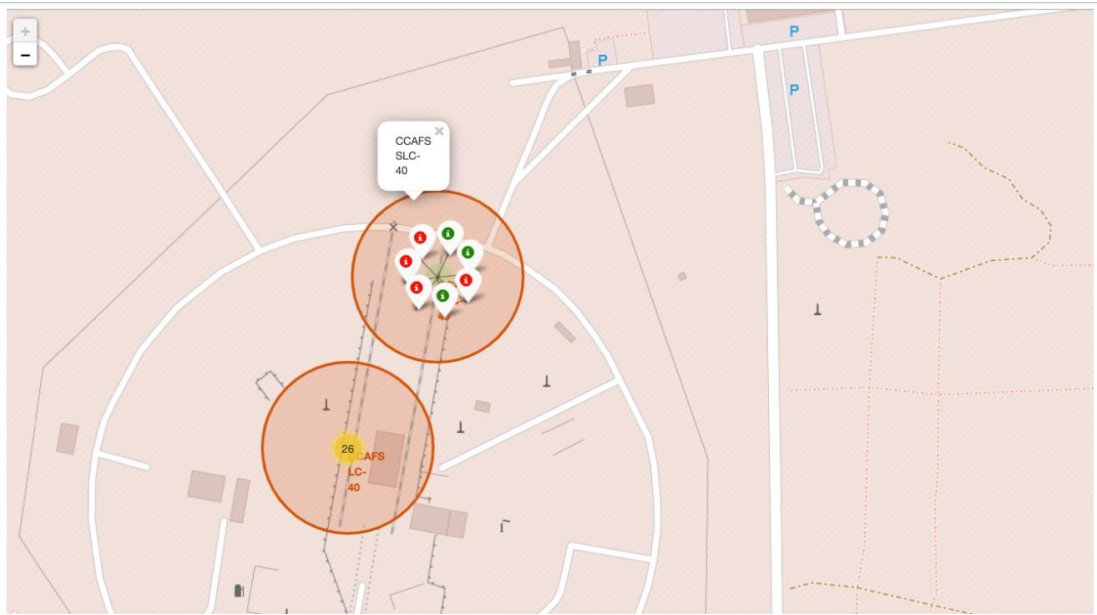
- SQL query results screenshots

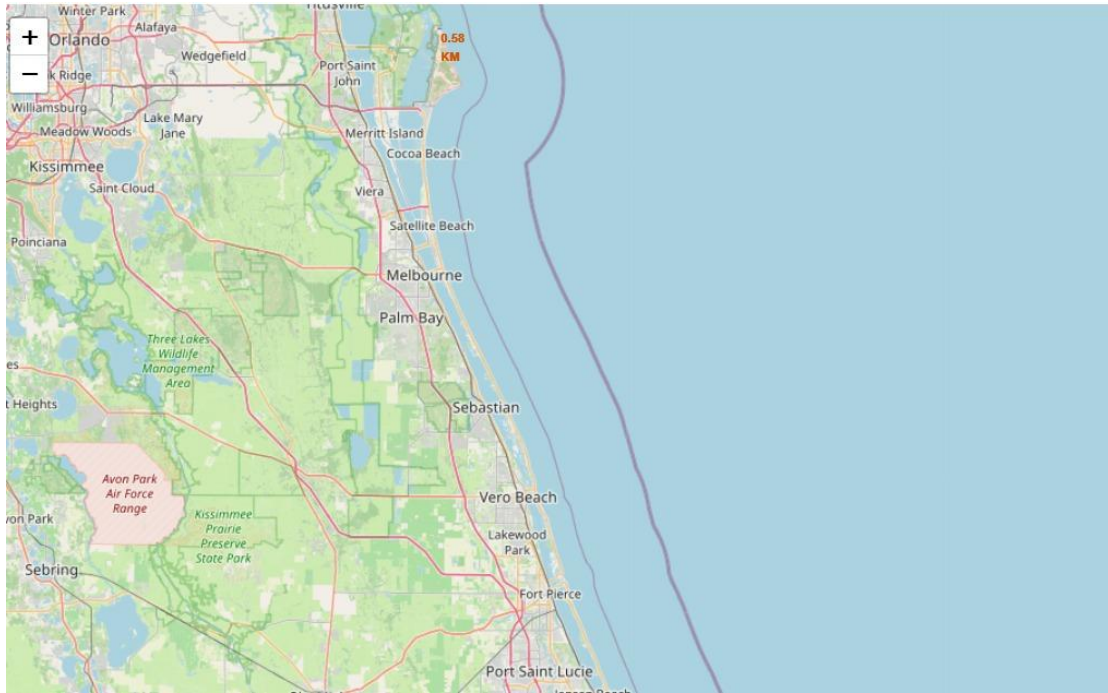
GitHub URL:

<https://github.com/khoraiankita/SpaceX-Falcon9-Landing-Prediction>

Interactive Analytics – Folium Maps







Map Visualizations

Launch Site Locations

Displayed all launch sites on an interactive map.

Launch Outcomes

Visualized successful and failed launches.

Proximity Analysis

Measured distances between launch sites and nearby infrastructure.

Findings

Geographic factors and launch site characteristics can influence mission operations.

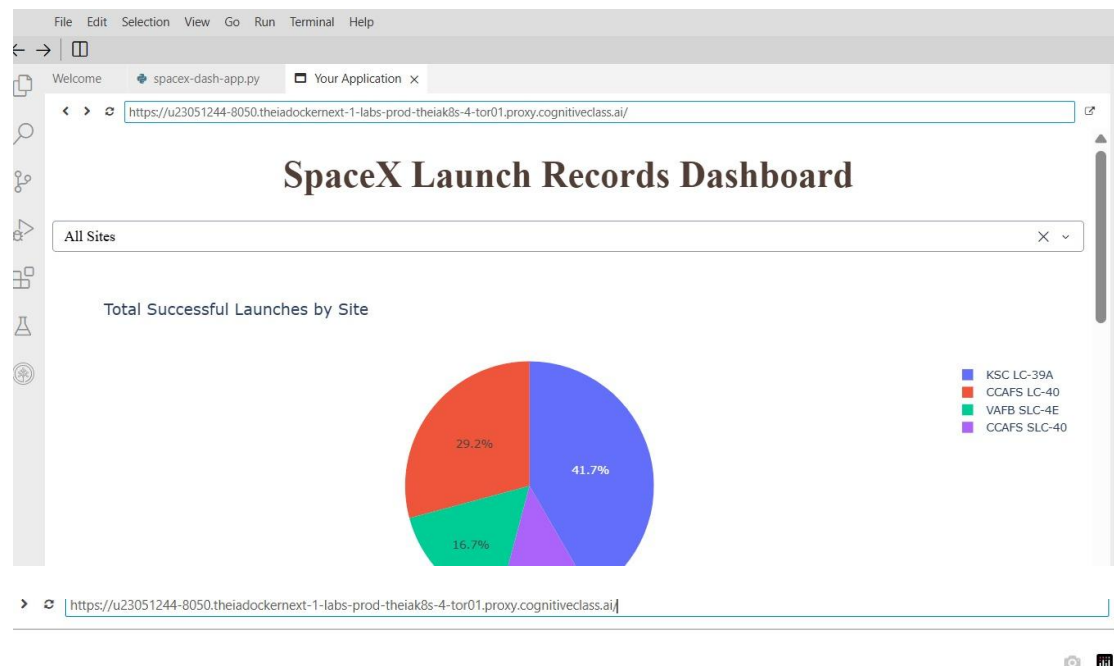
Include:

- Launch site markers
- Outcome markers
- Proximity analysis screenshots

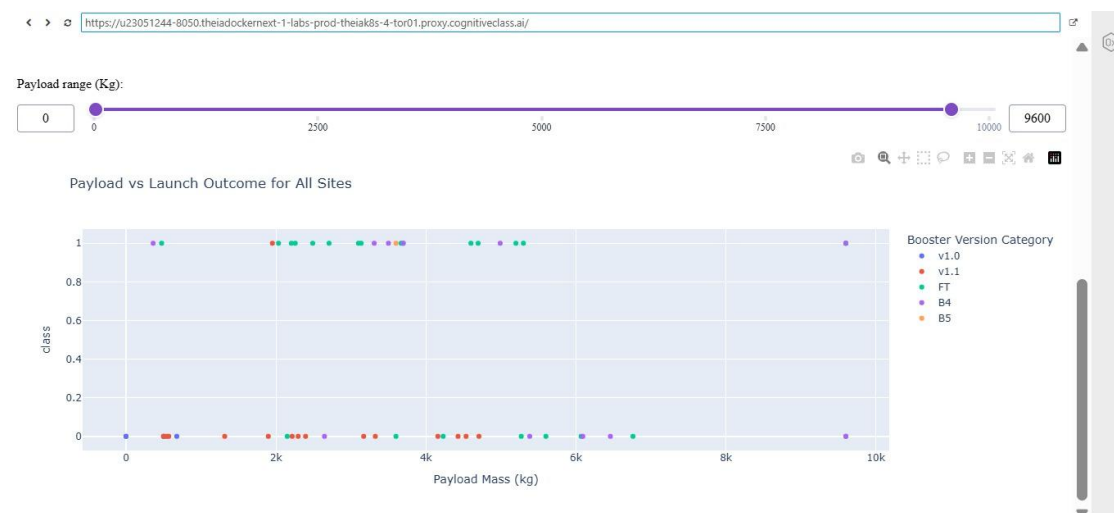
GitHub URL:

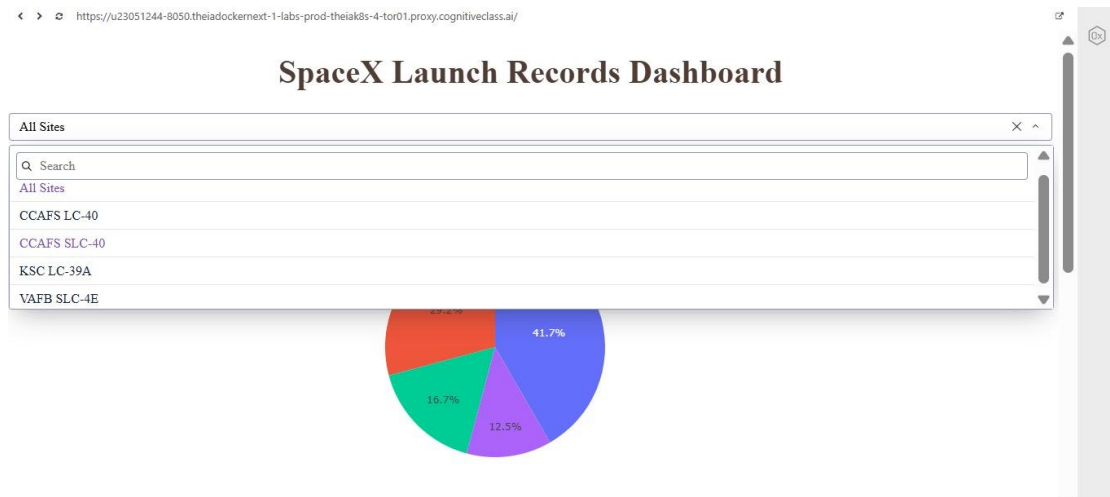
<https://github.com/khoriainkita/SpaceX-Falcon9-Landing-Prediction>

Interactive Analytics – Plotly Dash



Total Successful Launches by Site





Dashboard Features

Success Pie Chart

Visualized successful versus failed launches.

Payload vs Outcome Scatter Plot

Explored relationships between payload mass and landing success.

Benefits

Allows users to interactively explore launch performance data.

Include:

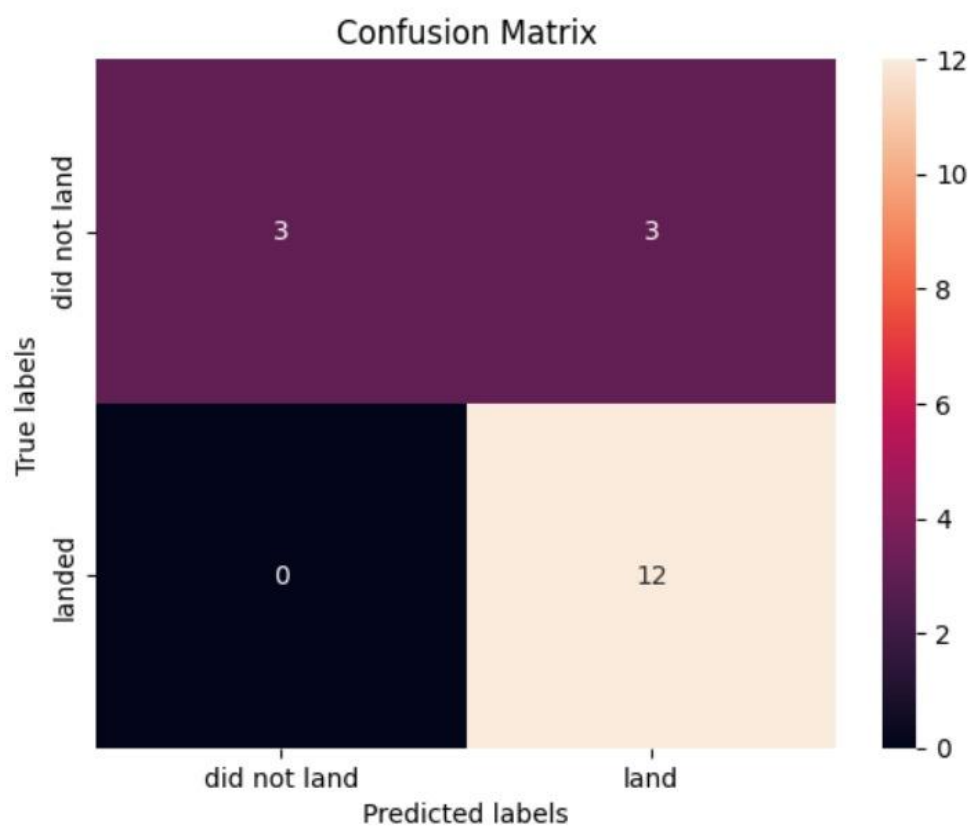
- Dashboard screenshots

GitHub URL:

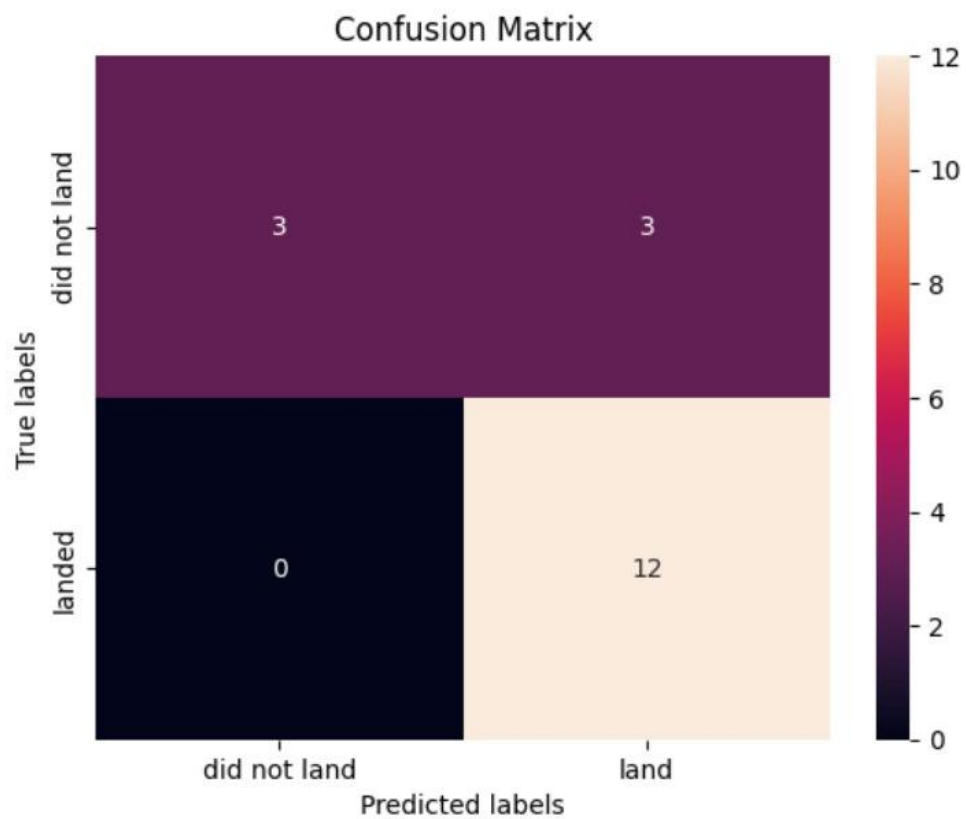
<https://github.com/khoraiankita/SpaceX-Falcon9-Landing-Prediction>

Predictive Analysis

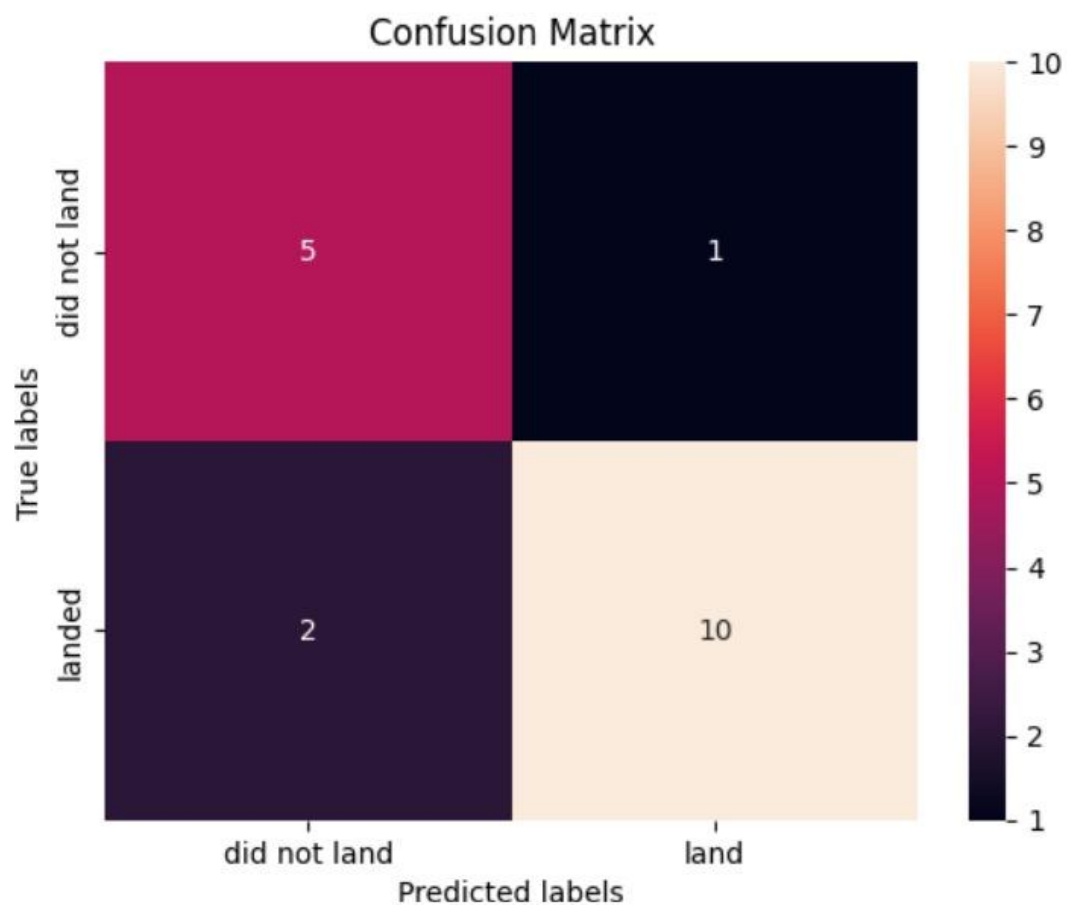
```
[19]: yhat=logreg_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```



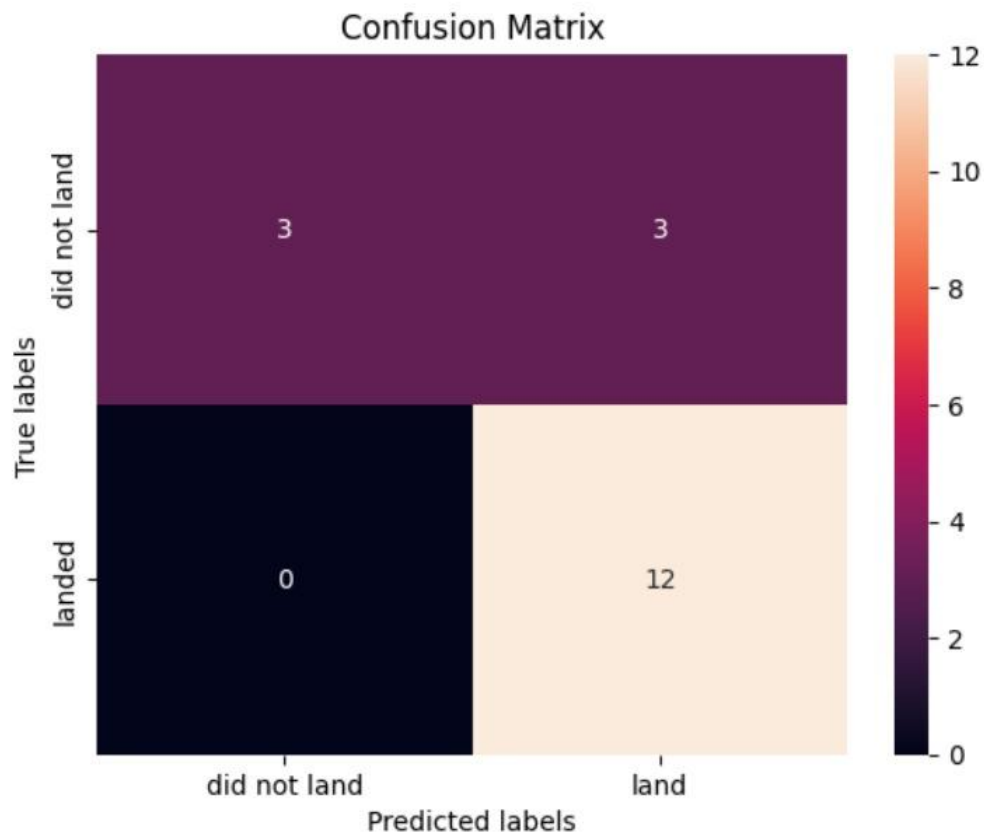
```
[26]: yhat=svm_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```



```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



```
[36]: yhat = knn_cv.predict(X_test)
      plot_confusion_matrix(Y_test,yhat)
```



```
[37]: print("Logistic Regression:", logreg_cv.score(X_test, Y_test))
      print("SVM:", svm_cv.score(X_test, Y_test))
      print("Decision Tree:", tree_cv.score(X_test, Y_test))
      print("KNN:", knn_cv.score(X_test, Y_test))
```

```
Logistic Regression: 0.8333333333333334
SVM: 0.8333333333333334
Decision Tree: 0.8333333333333334
KNN: 0.8333333333333334
```

Machine Learning Models

1. Logistic Regression
2. Support Vector Machine (SVM)
3. Decision Tree
4. K-Nearest Neighbors (KNN)

Evaluation Method

- Train/Test Split
- Cross Validation
- Accuracy Score

Include:

- Accuracy table for all models

Example:

Model	Accuracy
Logistic Regression	83.33%
SVM	83.33%
Decision Tree	83.33%
KNN	83.33%

Model Results and Best Model

Best Performing Model

[Insert your best model here]

Confusion Matrix

Include confusion matrix screenshot.

Interpretation

- True positives indicate correctly predicted successful landings.
- True negatives indicate correctly predicted failed landings.
- Low misclassification demonstrates strong predictive capability.

Conclusion

The selected model provides reliable predictions for Falcon 9 landing outcomes.

Final Conclusion

Summary

- Collected data from SpaceX API and Wikipedia.
- Performed data cleaning and preprocessing.
- Conducted EDA using visualizations and SQL.
- Developed interactive dashboards and maps.
- Trained multiple machine learning models.
- Identified the best model for landing prediction.

Future Improvements

- Incorporate weather conditions.
- Include additional mission parameters.
- Explore advanced ensemble learning methods.

Thank You

GitHub Repository:

<https://github.com/khoriainkita/SpaceX-Falcon9-Landing-Prediction>